# Local Semantic Search Engine — Specification v4

### A Distributed, Event-Driven, Local-First Knowledge Intelligence System

> This document is the single source of truth for code generation. Every section is implementation-ready. Claude Code should generate each service independently and respect all contracts defined here.

---

## 0. Purpose

A fully dockerized, multi-service knowledge system providing:

- Semantic + full-text search across files, emails, images, Confluence, and Jira
- RAG-based Q&A across your document corpus using a local LLM
- Dual translation pipeline (fast + high-quality), fully air-gapped
- Auto-summarization and entity extraction at ingestion time
- Proactive alerts and topic subscriptions
- Collaborative annotations indexed and searchable
- Expertise mapping based on reading behavior
- Related document surfacing
- Full admin configurability via system config panel
- Permission-based access control throughout
- Full observability stack

---

## 1. Repository Structure

```
repo/
├── docker-compose.yml
├── .env
├── migrations/              # Alembic DB migrations, one file per version
```

```
├── shared/
│   ├── models/              # Pydantic models shared across services
│   ├── schemas/             # JSON schemas for Kafka events
│   ├── utils/
│   │   ├── kafka.py
│   │   ├── logger.py        # Structured JSON logger with correlation_id
│   │   └── feature_flags.py # Reads system_config from DB
│   └── config/
│       └── settings.py      # Pydantic BaseSettings, reads from .env
│
├── services/
│   ├── api/
│   ├── ui/
│   ├── ingestion/
│   ├── worker-fast/
│   ├── worker-slow/
│   ├── worker-intelligence/  # NEW: summarization, entity extraction, alert matc
│   ├── scheduler/
│   └── preview/
│
└── infrastructure/
    ├── kafka/
    ├── elasticsearch/
    ├── qdrant/
    ├── postgres/
    ├── libretranslate/
    ├── ollama/              # NEW: local LLM
    ├── nifi/
    └── observability/
```

---

## 2. Environment Configuration (.env)

```
APP_ENV=dev

# Core
POSTGRES_URL=postgresql://postgres:postgres@postgres:5432/app
KAFKA_BROKER=kafka:9092
ELASTIC_URL=http://elasticsearch:9200
QDRANT_URL=http://qdrant:6333
FILES_ROOT=/data
JWT_SECRET=changeme

# Translation
LIBRETRANSLATE_URL=http://libretranslate:5000
```

```
# Local LLM (Ollama)
OLLAMA_URL=http://ollama:11434
OLLAMA_MODEL=mistral          # any model pulled in ollama service

# Auth
AUTH_PROVIDER=both            # local | ldap | both
LDAP_URL=ldap://domain-controller:389
LDAP_BASE_DN=DC=company,DC=local
LDAP_BIND_USER=cn=svc-search,DC=company,DC=local
LDAP_BIND_PASSWORD=changeme

# Feature defaults (overridden by system_config DB table at runtime)
FEATURE_RAG_QA=true
FEATURE_SUMMARIZATION=true
FEATURE_ENTITY_EXTRACTION=true
FEATURE_ANNOTATIONS=true
FEATURE_SUBSCRIPTIONS=true
FEATURE_EXPERTISE_MAP=true
FEATURE_RELATED_DOCS=true
FEATURE_AUTO_TAGGING=true
AUTO_ENRICH_THRESHOLD=5
INGEST_MODE=hybrid
```

> All FEATURE_* env vars are **default values only**. The `system_config` DB table overrides them at runtime. Admins change them via the admin panel without restarting services.

---

## 3. Docker Compose

Every service requires:

- `restart: unless-stopped`
- `healthcheck`
- `logging: driver: json-file`

### New services in v4

```
ollama:
  image: ollama/ollama:latest
  restart: unless-stopped
  volumes:
    - ollama_data:/root/.ollama
```

```
    environment:
      - OLLAMA_MODELS=/root/.ollama/models
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:11434/api/tags"]
    ports:
      - "11434:11434"
    # On first boot, pull model:
    # docker exec ollama ollama pull mistral

  worker-intelligence:
    build: ./services/worker-intelligence
    restart: unless-stopped
    depends_on:
      - kafka
      - postgres
      - ollama
      - qdrant
    environment:
      - POSTGRES_URL=${POSTGRES_URL}
      - KAFKA_BROKER=${KAFKA_BROKER}
      - OLLAMA_URL=${OLLAMA_URL}
      - OLLAMA_MODEL=${OLLAMA_MODEL}
      - QDRANT_URL=${QDRANT_URL}
    healthcheck:
      test: ["CMD", "python", "-c", "import requests; requests.get('http://localhos
```

---

# 4. Shared Models (Strict Contracts)

## 4.1 Kafka Topics

| Topic | Producer | Consumer | Purpose |
|---|---|---|---|
| `documents.raw` | ingestion | worker-fast | New/updated/deleted documents |
| `documents.enrichment` | api, worker-fast | worker-slow | Queue for high-quality translation |
| `documents.intelligence` | worker-fast | worker-intelligence | Queue for summarization + entity extraction + alert matching |
| `documents.dlq` | all | admin | Failed events after max retries |

| | | | |
|---|---|---|---|
| | workers | | |

### 4.2 Document Event (Kafka: `documents.raw`)

```
{
  "doc_id": "uuid",
  "source_id": "uuid",
  "source": "folder|nifi|confluence|jira",
  "path": "/data/file.pdf",
  "mime_type": "application/pdf",
  "source_language": "he",
  "operation": "create|update|delete",
  "timestamp": "ISO8601",
  "correlation_id": "uuid"
}
```

### 4.3 Intelligence Event (Kafka: `documents.intelligence`)

```
{
  "doc_id": "uuid",
  "content_english": "...",
  "group_id": "uuid",
  "correlation_id": "uuid",
  "tasks": ["summarize", "extract_entities", "match_alerts", "auto_tag"]
}
```

Only tasks enabled in `system_config` are included in `tasks[]`. Worker-intelligence skips tasks not in the list.

### 4.4 Indexed Document

```
{
  "doc_id": "...",
  "group_id": "...",
  "source_id": "...",
  "content_original": "...",
  "content_english": "...",
  "summary": "...",
  "tags": ["finance", "Q3", "procurement"],
  "translation_quality": "fast|high|null",
  "metadata": {
    "file_name": "...",
    "file_type": "...",
```

```
    "size": 12345,
    "created_at": "...",
    "source_language": "he",
    "target_language": "en"
  }
}
```

---

# 5. Database Schema (Complete)

## 5.1 Users & Auth

```
CREATE TABLE users (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         TEXT UNIQUE NOT NULL,
  display_name  TEXT,
  auth_source   TEXT NOT NULL CHECK (auth_source IN ('local', 'ldap')),
  password_hash TEXT,               -- bcrypt; NULL for LDAP users
  is_admin      BOOLEAN NOT NULL DEFAULT false,
  created_at    TIMESTAMPTZ DEFAULT now(),
  last_login    TIMESTAMPTZ
);
```

## 5.2 Groups & Permissions

```
CREATE TABLE groups (
  id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT UNIQUE NOT NULL
);

CREATE TABLE user_groups (
  user_id  UUID REFERENCES users(id) ON DELETE CASCADE,
  group_id UUID REFERENCES groups(id) ON DELETE CASCADE,
  PRIMARY KEY (user_id, group_id)
);

CREATE TABLE source_permissions (
  source_id UUID NOT NULL,
  group_id  UUID REFERENCES groups(id) ON DELETE CASCADE,
  PRIMARY KEY (source_id, group_id)
);
```

### 5.3 Ingestion Sources

```
CREATE TABLE ingestion_sources (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            TEXT NOT NULL,
  type            TEXT NOT NULL CHECK (type IN ('folder', 'nifi', 'confluence', '
  path            TEXT,
  source_language TEXT NOT NULL,
  enabled         BOOLEAN DEFAULT true,
  config          JSONB
  -- folder:     { watch_recursive: bool }
  -- confluence: { base_url, api_token, username, spaces: [], poll_cron }
  -- jira:       { base_url, api_token, username, projects: [], poll_cron }
);

CREATE TABLE atlassian_sync_state (
  source_id    UUID PRIMARY KEY REFERENCES ingestion_sources(id),
  last_sync_at TIMESTAMPTZ,
  last_cursor  TEXT
);
```

### 5.4 Activity & View Counts

```
CREATE TABLE user_activity (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id        UUID REFERENCES users(id) ON DELETE CASCADE,
  action         TEXT NOT NULL CHECK (action IN ('search', 'read', 'qa', 'annotat
  doc_id         UUID,
  query          TEXT,
  correlation_id TEXT,
  created_at     TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON user_activity (user_id, created_at DESC);
CREATE INDEX ON user_activity (doc_id) WHERE doc_id IS NOT NULL;

CREATE TABLE document_view_counts (
  doc_id            UUID PRIMARY KEY,
  view_count        INTEGER NOT NULL DEFAULT 0,
  last_viewed       TIMESTAMPTZ,
  enrichment_queued BOOLEAN NOT NULL DEFAULT false
);
```

### 5.5 Document Intelligence (NEW)

```
-- Auto-generated summary per document
CREATE TABLE document_summaries (
  doc_id     UUID PRIMARY KEY,
  summary    TEXT NOT NULL,
  model      TEXT NOT NULL,   -- e.g. "mistral"
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Named entity registry (deduped by name + type)
CREATE TABLE entities (
  id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  type TEXT NOT NULL CHECK (type IN ('person', 'organization', 'location', 'date'
  UNIQUE (name, type)
);

-- Document ↔ entity many-to-many
CREATE TABLE document_entities (
  doc_id    UUID NOT NULL,
  entity_id UUID REFERENCES entities(id) ON DELETE CASCADE,
  frequency INTEGER DEFAULT 1,
  PRIMARY KEY (doc_id, entity_id)
);
CREATE INDEX ON document_entities (entity_id);

-- Auto-assigned topic tags
CREATE TABLE document_tags (
  doc_id UUID NOT NULL,
  tag    TEXT NOT NULL,
  PRIMARY KEY (doc_id, tag)
);
CREATE INDEX ON document_tags (tag);
```

## 5.6 Annotations (NEW)

```
CREATE TABLE annotations (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  doc_id     UUID NOT NULL,
  user_id    UUID REFERENCES users(id) ON DELETE CASCADE,
  text       TEXT NOT NULL,          -- the highlighted passage
  note       TEXT,                   -- user's comment on the highlight
  position   JSONB,                  -- { page, start_char, end_char } — format d
  is_private BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now(),
```

```
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON annotations (doc_id);
CREATE INDEX ON annotations (user_id);
-- Annotations are indexed in Elasticsearch under a separate index: "annotations"
```

### 5.7 Alert Subscriptions (NEW)

```
CREATE TABLE alert_subscriptions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID REFERENCES users(id) ON DELETE CASCADE,
  query           TEXT NOT NULL,           -- natural language topic e.g. "procur
  similarity_threshold FLOAT DEFAULT 0.75, -- cosine similarity cutoff for matchi
  enabled         BOOLEAN DEFAULT true,
  created_at      TIMESTAMPTZ DEFAULT now(),
  last_notified   TIMESTAMPTZ
);

CREATE TABLE alert_notifications (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subscription_id UUID REFERENCES alert_subscriptions(id) ON DELETE CASCADE,
  user_id         UUID REFERENCES users(id) ON DELETE CASCADE,
  doc_id          UUID NOT NULL,
  similarity      FLOAT NOT NULL,
  read            BOOLEAN DEFAULT false,
  created_at      TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON alert_notifications (user_id, read, created_at DESC);
```

### 5.8 System Config (NEW — admin-configurable feature flags)

```
CREATE TABLE system_config (
  key        TEXT PRIMARY KEY,
  value      JSONB NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now(),
  updated_by UUID REFERENCES users(id)
);

-- Seed values (insert on first migration):
INSERT INTO system_config (key, value) VALUES
  ('feature.rag_qa',              'true'),
  ('feature.summarization',       'true'),
  ('feature.entity_extraction',   'true'),
  ('feature.annotations',         'true'),
```

```
  ('feature.subscriptions',       'true'),
  ('feature.expertise_map',       'true'),
  ('feature.related_docs',        'true'),
  ('feature.auto_tagging',        'true'),
  ('llm.model',                   '"mistral"'),
  ('llm.qa_system_prompt',        '"You are a knowledge assistant. Answer based o
  ('llm.summarization_prompt',    '"Summarize the following document in 3-5 sente
  ('llm.entity_extraction_prompt','"Extract named entities (people, organizations
  ('llm.auto_tag_prompt',         '"Assign 3-7 short topic tags to the following
  ('search.vector_weight',        '0.7'),
  ('search.bm25_weight',          '0.3'),
  ('search.related_docs_limit',   '5'),
  ('auto_enrich.threshold',       '5'),
  ('alerts.similarity_threshold', '0.75'),
  ('alerts.check_on_ingest',      'true');
```

> All services read `system_config` via `shared/utils/feature_flags.py`. Flags are cached for 60 seconds to avoid per-request DB hits. Changing a flag in the admin panel takes effect within 60 seconds system-wide without restart.

---

# 6. API Service (FastAPI)

## 6.1 Structure

```
api/
├── main.py
├── routers/
│   ├── auth.py
│   ├── search.py
│   ├── qa.py               # NEW: RAG Q&A
│   ├── preview.py
│   ├── download.py
│   ├── annotations.py      # NEW
│   ├── subscriptions.py    # NEW
│   ├── activity.py
│   ├── expertise.py        # NEW
│   └── admin.py
├── services/
│   ├── enrichment.py       # translation request + auto-enrich logic
│   ├── rag.py              # NEW: RAG pipeline (retrieve → assemble → generate)
│   ├── alert_service.py    # NEW: subscription management
│   └── feature_flags.py    # reads system_config
├── models/
```

```
├── permissions/
│   └── enforcer.py
└── middleware/
    └── correlation.py
```

## 6.2 Activity Logging

Every `search`, `read`, `qa`, and `annotate` action is logged to `user_activity`. On every `read`:

1. Insert `user_activity`
2. Increment `document_view_counts.view_count`
3. If `view_count >= system_config['auto_enrich.threshold']` AND `translation_quality != 'high'` AND `enrichment_queued = false`:
   - Publish to `documents.enrichment`
   - Set `enrichment_queued = true`

## 6.3 All Endpoints

### Auth

```
POST /auth/login
POST /auth/logout
GET  /auth/me
```

### Search

```
POST /search
Body: { query, filters?: { tags, source_type, date_from, date_to }, page, page_si
Returns: { results: [{ doc_id, title, summary, score, tags, translation_quality,
```

- Permission-filtered by user groups
- Hybrid BM25 + vector with weights from `system_config`
- Returns `summary` from `document_summaries` if available

### Q&A (feature-flagged: `feature.rag_qa`)

```
POST /qa
Body: { question, group_filter?: [group_ids] }
Returns: {
  answer: "...",
  citations: [{ doc_id, title, chunk, score }],
  model: "mistral",
  latency_ms: 420
}
```

- Fails gracefully with 503 if Ollama is unreachable
- Logs to `user_activity` with `action='qa'`

### Preview

```
GET /preview/{doc_id}
Returns: { doc_id, preview_mode, content, content_type, summary?, annotations?, e
```

### Download

```
GET /download/{doc_id}
```

### Related Documents (feature-flagged: `feature.related_docs`)

```
GET /doc/{doc_id}/related
Returns: [{ doc_id, title, score, summary }]   # top N from system_config
```

### Translation

```
POST /doc/{doc_id}/request-translation
Returns: 200 (queued) | 202 (already queued) | 204 (already high quality)
```

### Annotations (feature-flagged: `feature.annotations`)

```
GET    /doc/{doc_id}/annotations          # own + shared annotations on this doc
POST   /doc/{doc_id}/annotations          # create
PUT    /annotations/{id}                  # update note or is_private
DELETE /annotations/{id}                  # own annotations only; admin can delet
```

### Subscriptions (feature-flagged: `feature.subscriptions`)

```
GET    /subscriptions                      # list own subscriptions
POST   /subscriptions                      # create
PUT    /subscriptions/{id}                 # update query or threshold
DELETE /subscriptions/{id}                 # delete
GET    /notifications                      # unread alerts
PUT    /notifications/{id}/read
```

### Activity / History

```
GET /activity/me?action=&from=&to=&limit=&offset=
```

### Expertise Map (feature-flagged: `feature.expertise_map`)

```
GET /expertise?topic=procurement          # users who read most docs matching topi
Returns: [{ user_id, display_name, read_count, top_docs }]
```

### Admin

```
# Users
GET    /admin/users
POST   /admin/users
PUT    /admin/users/{id}
DELETE /admin/users/{id}

# Groups & Permissions
GET    /admin/groups
POST   /admin/groups
POST   /admin/permissions

# Sources
GET    /admin/ingestion
POST   /admin/ingestion
PUT    /admin/ingestion/{id}
DELETE /admin/ingestion/{id}

# Atlassian
POST   /admin/atlassian/{source_id}/sync-now
GET    /admin/atlassian/{source_id}/sync-state

# DLQ
```

```
GET    /admin/dlq
POST   /admin/dlq/{id}/retry

# Activity audit
GET    /admin/activity?user_id=&action=&from=&to=

# System Config (feature flags + LLM settings + search weights)
GET    /admin/config                      # returns all system_config rows
PUT    /admin/config/{key}               # update single config value
POST   /admin/config/reset               # reset all to seed defaults

# Entities (read-only view)
GET    /admin/entities?type=&search=
GET    /admin/entities/{id}/documents

# Delete
DELETE /doc/{doc_id}                     # admin only
```

### 6.4 Permission Enforcement

```
# api/permissions/enforcer.py

def require_admin(user: TokenPayload):
    if not user.is_admin:
        raise HTTPException(403)

def require_feature(flag: str):
    if not get_feature_flag(flag):
        raise HTTPException(404, detail=f"Feature '{flag}' is disabled")

def get_allowed_groups(user: TokenPayload) -> list[str]:
    return user.groups

# Used on every search/preview/download endpoint:
def assert_doc_access(doc: Document, user: TokenPayload):
    if doc.group_id not in user.groups:
        raise HTTPException(403)
```

---

## 7. Auth Service

### 7.1 Strategy

- `AUTH_PROVIDER=both` : LDAP attempted first, local DB fallback

- LDAP login: upsert user, sync AD groups → `user_groups`
- Local login: bcrypt verify, no external call
- Both paths issue identical JWT

### 7.2 JWT Payload

```
{
  "sub": "user-uuid",
  "email": "user@company.com",
  "is_admin": false,
  "groups": ["group-uuid-1", "group-uuid-2"],
  "auth_source": "ldap|local",
  "exp": 1234567890
}
```

Token TTL: 8 hours. No refresh tokens in v1.

---

## 8. Ingestion Service

### 8.1 Responsibilities

- Load source configs from Postgres on startup, reload every 60s
- Folder sources: start `watchdog` observer per source
- NiFi sources: Kafka consumer on `documents.raw`
- Atlassian sources: registered with scheduler, polled daily
- Dedup via `sha256` in `ingested_files`
- Publish `DocumentEvent` to `documents.raw`

### 8.2 Deduplication

```
CREATE TABLE ingested_files (
  sha256      TEXT PRIMARY KEY,
  doc_id      UUID NOT NULL,
  source_id   UUID NOT NULL,
  ingested_at TIMESTAMPTZ DEFAULT now()
);
```

### 8.3 Atlassian Polling (Server/Data Center only)

Reject any `base_url` matching `*.atlassian.net` at source creation. Poll schedule: daily at 02:00 (default), overridable per source via `config.poll_cron`.

**Confluence flow:** Fetch pages updated since `last_sync_at`. Extract HTML body, strip to plain text. Fetch attachments as separate events. Upsert `atlassian_sync_state`.

**Jira flow:** Fetch issues updated since `last_sync_at` via JQL. Concatenate summary + description + comments as content. Fetch attachments as separate events.

**Content mapping:**

| Object | **`doc_id` key** | **`mime_type`** |
|---|---|---|
| Confluence page | `confluence:{page_id}` | `text/html` |
| Confluence attachment | `confluence:{page_id}:att:{att_id}` | original |
| Jira issue | `jira:{issue_key}` | `text/plain` |
| Jira attachment | `jira:{issue_key}:att:{att_id}` | original |

---

## 9. Fast Worker

### 9.1 Pipeline

```
consume(documents.raw)
  → extract_text()          # tika / pdfminer / custom per mime_type
  → ocr_if_needed()         # tesseract fallback for image PDFs
  → detect_language()       # only if source_language is null
  → translate_fast()        # LibreTranslate → English; fallback: index untranslat
  → chunk()                 # 512 tokens, 50 overlap, sentence boundary
  → embed()                 # batch via sentence-transformers (min 32/batch)
  → index_elastic()         # BM25 on content_english
  → index_qdrant()          # vector per chunk, payload: { doc_id, group_id, chunk
  → publish_intelligence()  # → documents.intelligence (tasks from system_config)
  → mark_complete()
```

### 9.2 **`publish_intelligence()`**

```
def publish_intelligence(doc_id, content_english, group_id):
```

```
    tasks = []
    if get_flag("feature.summarization"):      tasks.append("summarize")
    if get_flag("feature.entity_extraction"):  tasks.append("extract_entities")
    if get_flag("feature.auto_tagging"):       tasks.append("auto_tag")
    if get_flag("alerts.check_on_ingest"):     tasks.append("match_alerts")
    if tasks:
        publish("documents.intelligence", { doc_id, content_english, group_id, ta
```

### 9.3 Translation Fallback

On LibreTranslate failure: index with `content_english = content_original`, set `translation_quality = null`, push to `documents.enrichment` for slow worker retry.

---

## 10. Worker Intelligence (NEW SERVICE)

This service consumes `documents.intelligence` and runs all LLM-powered tasks via Ollama.

### 10.1 Structure

```
worker-intelligence/
├── main.py
├── tasks/
│   ├── summarize.py
│   ├── extract_entities.py
│   ├── auto_tag.py
│   └── match_alerts.py
└── ollama_client.py        # wrapper around Ollama HTTP API
```

### 10.2 Task: Summarize

```
def summarize(doc_id: str, content: str):
    prompt = get_config("llm.summarization_prompt") + "\n\n" + content[:8000]
    summary = ollama_generate(prompt)
    upsert("document_summaries", { doc_id, summary, model: OLLAMA_MODEL })
    # Also update Elasticsearch document with summary field
```

### 10.3 Task: Extract Entities

```
def extract_entities(doc_id: str, content: str):
    prompt = get_config("llm.entity_extraction_prompt") + "\n\n" + content[:6000]
```

```
    result = ollama_generate(prompt)
    entities = parse_json(result)   # [{ name, type }]
    for entity in entities:
        entity_id = upsert_entity(entity.name, entity.type)
        upsert("document_entities", { doc_id, entity_id })
```

### 10.4 Task: Auto Tag

```
def auto_tag(doc_id: str, content: str):
    prompt = get_config("llm.auto_tag_prompt") + "\n\n" + content[:4000]
    tags = parse_json(ollama_generate(prompt))  # ["finance", "Q3", ...]
    replace("document_tags", doc_id, tags)
    # Update Elasticsearch document tags[] field
```

### 10.5 Task: Match Alerts

```
def match_alerts(doc_id: str, content: str, group_id: str):
    doc_vector = embed(content)
    subscriptions = get_active_subscriptions()  # from DB
    for sub in subscriptions:
        # Only match if user has access to this document's group
        if group_id not in get_user_groups(sub.user_id):
            continue
        sub_vector = embed(sub.query)
        similarity = cosine(doc_vector, sub_vector)
        threshold = sub.similarity_threshold or get_config("alerts.similarity_thr
        if similarity >= threshold:
            insert("alert_notifications", {
                subscription_id: sub.id,
                user_id: sub.user_id,
                doc_id,
                similarity
            })
```

### 10.6 Ollama Client

```
# ollama_client.py
import httpx

def ollama_generate(prompt: str, model: str = None) -> str:
    model = model or get_config("llm.model")
    response = httpx.post(f"{OLLAMA_URL}/api/generate", json={
        "model": model,
```

```
        "prompt": prompt,
        "stream": False
    }, timeout=120)
    response.raise_for_status()
    return response.json()["response"]
```

### 10.7 Error Handling

If Ollama is unavailable:

- Log structured warning with `correlation_id`
- Skip all LLM tasks for this event
- Do **not** push to DLQ — intelligence tasks are best-effort and non-blocking
- Failed tasks are retried on next document update

---

## 11. RAG Q&A Pipeline (api/services/rag.py)

```
async def answer_question(question: str, user: TokenPayload) -> QAResponse:
    # 1. Retrieve relevant chunks
    query_vector = embed(question)
    chunks = qdrant.search(
        query_vector=query_vector,
        filter=group_id IN user.groups,
        limit=8
    )

    # 2. Assemble context
    context = "\n\n---\n\n".join([
        f"Source: {c.doc_id}\n{c.text}" for c in chunks
    ])

    # 3. Build prompt
    system_prompt = get_config("llm.qa_system_prompt")
    prompt = f"{system_prompt}\n\nContext:\n{context}\n\nQuestion: {question}\nAn

    # 4. Generate
    answer = ollama_generate(prompt)

    # 5. Log activity
    insert_activity(user.id, action='qa', query=question)
```

```
    return QAResponse(
        answer=answer,
        citations=[{ doc_id: c.doc_id, chunk: c.text, score: c.score } for c in c
        model=get_config("llm.model")
    )
```

---

# 12. Slow Worker

## 12.1 Pipeline

```
fetch_docs_to_enrich()       # WHERE translation_quality IS NULL OR = 'fast'
  → translate_high_quality() # LibreTranslate in v1; pluggable interface for futu
  → re_embed()
  → update_elastic()
  → update_qdrant()
  → set translation_quality = 'high'
  → publish_intelligence()   # re-run LLM tasks on improved translation
```

## 12.2 Triggered by

- Scheduler: nightly at 03:00
- Kafka: `documents.enrichment` topic (manual requests + auto-enrich)

---

# 13. Preview Service

## 13.1 File Type → Output

| File type | `preview_mode` | Output |
|---|---|---|
| docx / odt | `html` | LibreOffice headless → HTML |
| xlsx / csv | `table` | JSON: `{ headers: [], rows: [[]] }` |
| pptx | `slides` | JSON: `[{ index, title, content, notes }]` |
| pdf (text) | `html` | Text layer → HTML |
| pdf (scanned) | `image` | pdftoppm → base64 image |

| | | |
|---|---|---|
| images | `image` | Served as-is |
| zip / tar | `archive` | File listing JSON |
| eml / msg | `email` | HTML + `{ attachments: [] }` |
| Confluence page | `html` | body.storage → HTML |
| Jira issue | `html` | Structured HTML (summary + description + comments) |
| plaintext | `text` | Raw text |

### 13.2 API

```
GET /preview/{doc_id}
GET /archive/{doc_id}/contents
GET /archive/{doc_id}/file?path=...
```

### 13.3 Response Schema

```
{
  "doc_id": "...",
  "preview_mode": "html|table|slides|image|text|archive|email",
  "content": "...",
  "content_type": "text/html|application/json|...",
  "summary": "...",
  "tags": [],
  "entities": [{ "name": "...", "type": "..." }],
  "annotations": [{ "id", "user_display_name", "text", "note", "position", "is_pr
  "translation_quality": "fast|high|null",
  "related": [{ "doc_id", "title", "score" }]
}
```

---

## 14. Search Implementation

### Elasticsearch (BM25)

```
{
  "query": {
```

```
    "bool": {
      "must": { "multi_match": {
        "query": "<query>",
        "fields": ["content_english", "summary^1.5", "tags^2"]
      }},
      "filter": { "terms": { "group_id": ["<user_groups>"] } }
    }
  }
}
```

### Qdrant (Vector)

```
client.search(
    collection_name="documents",
    query_vector=embed(query),
    query_filter=Filter(must=[
        FieldCondition(key="group_id", match=MatchAny(any=user.groups))
    ]),
    limit=50
)
```

### Merge

```
vector_weight = float(get_config("search.vector_weight"))   # default 0.7
bm25_weight   = float(get_config("search.bm25_weight"))     # default 0.3
score = vector_weight * vector_score + bm25_weight * bm25_score
```

### Related Documents

```
def get_related(doc_id: str, user: TokenPayload):
    doc_vector = qdrant.get_vector(doc_id)
    return qdrant.search(
        query_vector=doc_vector,
        query_filter=group_id IN user.groups AND doc_id != doc_id,
        limit=get_config("search.related_docs_limit")
    )
```

---

## 15. Chunking Strategy

- Size: 512 tokens, 50-token overlap

- Split on sentence boundaries; hard cut at token limit
- Uniform across all file types
- Each chunk = one Qdrant point with payload `{ doc_id, group_id, chunk_index, text }`
- Elasticsearch indexes full `content_english` per document (not per chunk)

---

## 16. Translation Strategy

| Property | Value |
|---|---|
| Backend | LibreTranslate (self-hosted, air-gapped) |
| Target language | English always |
| Source language | `ingestion_sources.source_language` ; null = auto-detect |
| Fast quality | LibreTranslate default |
| High quality | Pluggable — LibreTranslate in v1, swappable without spec change |

---

## 17. UI (Next.js)

### Pages

| Route | Description | Auth |
|---|---|---|
| `/` | Search | required |
| `/qa` | Ask a question (RAG) | required, feature-flagged |
| `/doc/[id]` | Document preview | required |
| `/history` | Personal search + read history | required |
| `/subscriptions` | Manage alert subscriptions | required, feature-flagged |
| `/notifications` | Unread alerts | required, feature-flagged |
| | | |

| | | |
|---|---|---|
| `/expertise` | Expertise map browser | required, feature-flagged |
| `/admin` | Admin panel | admin only |

## Components

### Search & Results

- `SearchBar` — keyword + semantic search input
- `ResultsList` — results with summary snippet, tags, source badge, translation quality indicator
- `FacetSidebar` — filter by tag, source type, date range

### Document Preview

- `PreviewPanel` — renders per `preview_mode`
- `SummaryBadge` — shows auto-generated summary
- `TranslationBadge` — `fast` / `high` with "Request better" button
- `AnnotationSidebar` — create/view annotations on the document; toggle private/shared
- `RelatedDocsList` — "You might also need" panel (feature-flagged)
- `EntityList` — named entities extracted from this document

### Q&A

- `QAInput` — question input with submit
- `QAAnswer` — answer text with citation cards linking to source documents
- `QAHistory` — past questions in current session

### Intelligence

- `NotificationBell` — unread alert count in nav
- `NotificationsList` — list of alerts with document links
- `SubscriptionManager` — create/edit/delete subscriptions with threshold slider
- `ExpertiseMap` — topic input → list of users sorted by relevance

### Admin Panel

- `UserManager` — create/edit/delete local users, assign groups

- `GroupManager` — create groups, manage memberships
- `SourceManager` — configure ingestion sources with language and type
- `PermissionsMatrix` — map sources to groups
- `DLQViewer` — failed events table with retry button
- `ActivityLog` — filterable audit log
- `SystemConfigEditor` — toggle feature flags, edit LLM prompts, tune search weights, set thresholds — all live, no restart required
- `AtlassianSyncStatus` — last sync time, sync-now button, error state per source
- `EntityBrowser` — browse extracted entities, view related documents

---

## 18. Permissions

All permission logic lives exclusively in `api/permissions/enforcer.py`.

```
def require_admin(user):       raise 403 if not user.is_admin
def require_feature(flag):     raise 404 if feature disabled
def assert_doc_access(doc, user): raise 403 if doc.group_id not in user.groups
def assert_annotation_owner(annotation, user):
    if annotation.user_id != user.sub and not user.is_admin:
        raise HTTPException(403)
```

**Delete rules:**

- `DELETE /doc/{doc_id}` — admin only, enforced at router level
- `DELETE /annotations/{id}` — own annotations only; admin can delete any
- Workers never publish delete events

---

## 19. Dead-Letter Queue

- **Topic:** `documents.dlq`
- **Payload:** `{ original_event, error, retry_count, failed_at }`
- **Max retries before DLQ:** 3 (exponential backoff: 5s, 30s, 120s)

- **Intelligence task failures:** not sent to DLQ (best-effort, logged only)
- **Admin:** `GET /admin/dlq`, `POST /admin/dlq/{id}/retry`

---

## 20. Delete Cascade

`operation: delete` consumed by fast worker:

1. Delete from Elasticsearch
2. Delete from Qdrant (all chunks)
3. Delete from `document_summaries`, `document_entities`, `document_tags`, `document_view_counts`
4. Delete from `ingested_files`

Annotations are **retained** on delete — they may contain valuable user knowledge. Marked with `doc_deleted: true`.

**Consistency:** Best-effort. Each step logged with `correlation_id`.

---

## 21. Observability

- **Logs:** JSON, `correlation_id` on every line, all services
- **Metrics:** `/metrics` on all services (Prometheus format)
- **Stack:** Prometheus + Grafana + Loki
- **Key metrics to expose per service:**
  - `documents_processed_total` (fast/slow worker)
  - `llm_request_duration_seconds` (worker-intelligence)
  - `search_request_duration_seconds` (api)
  - `qa_request_duration_seconds` (api)
  - `dlq_depth` (scheduler)
  - `alert_notifications_created_total` (worker-intelligence)

---

## 22. Security

- JWT signed with `JWT_SECRET`; all enforcement backend-only
- Delete: admin claim required at router level (not business logic)
- All HTML output: DOMPurify (frontend) + bleach (backend)
- Annotations: shared annotations visible to anyone with doc access; private annotations visible to owner only
- LLM prompts in `system_config` are admin-editable — treat as trusted input
- Ollama, LibreTranslate: no external calls, fully local
- Atlassian Cloud URLs (`*.atlassian.net`): rejected at source creation

---

## 23. Error Handling

| Service | Retry | DLQ on failure | Notes |
|---|---|---|---|
| worker-fast | 3× exponential | Yes | Translation failure → index untranslated + enrichment queue |
| worker-slow | 3× exponential | Yes | |
| worker-intelligence | 0 (skip) | No | Best-effort; logged, not blocking |
| ingestion | 3× exponential | Yes | |
| api → Ollama | 0 | No | Returns 503 with user-friendly message |

---

## 24. Build Order

1. Postgres schema + Alembic migrations (all tables including `system_config` seed)
2. Auth service (local + LDAP)
3. API skeleton (search, permissions, admin, system config endpoints)

4. Fast worker (without `publish_intelligence`)
5. Basic UI (search + preview + admin panel with config editor)
6. Preview service
7. Slow worker
8. Worker intelligence (Ollama: summarize → entity extract → auto-tag → alert match)
9. Q&A endpoint + UI
10. Annotations
11. Subscriptions + notifications
12. Expertise map
13. Related documents
14. Observability
15. NiFi integration
16. Atlassian integration (Confluence + Jira Server/DC)

> Steps 1–7 produce a fully working search system. Steps 8–13 add the intelligence layer. Steps 14–16 add ops and external integrations.

---

## 25. Non-Functional Requirements

| Requirement | Target |
|---|---|
| Document capacity | 500K+ |
| Ingestion latency | < 5s |
| Search latency | < 300ms |
| Q&A latency | < 10s (LLM-bound) |
| Summarization latency | < 30s per doc (background) |
| Data loss | Zero (intelligence tasks are best-effort) |
| Air-gap | Full — no external APIs in v1 |

| | |
|---|---|
| Feature flag propagation | < 60s after admin change |

---

## 26. Resolved Decisions

| Topic | Decision |
|---|---|
| Translation backend | LibreTranslate (self-hosted) |
| Target language | English always |
| Source language | Per ingestion source in DB |
| Chunking | 512 tokens / 50 overlap / sentence boundary |
| Auth | AD + local, unified Postgres user table |
| User overlap | Single record, `auth_source` tracks last method |
| Delete permission | Admin only, router-level enforcement |
| Delete cascade | Best-effort, logged; annotations retained |
| DLQ | `documents.dlq`, admin-managed retry |
| Intelligence failures | Best-effort, no DLQ, logged only |
| Preview modes | `html`, `table`, `slides`, `image`, `text`, `archive`, `email` |
| Atlassian | Server/Data Center only, daily poll, manual group mapping |
| Activity tracking | `user_activity` + `document_view_counts` |
| Auto-enrich threshold | 5 views, configurable in system_config |
| LLM | Ollama (local), model configurable in system_config |
| Feature flags | `system_config` table, < 60s propagation, editable via admin panel |
| RAG context window | Top 8 chunks by cosine similarity, permission-filtered |
| Alert matching | Cosine similarity at ingest time, threshold configurable per subscription |
| Annotation visibility | Private (owner only) or shared (anyone with doc access) |

| | |
|---|---|
| Expertise map | Based on `user_activity` read history, topic matched via vector search |

---

*END OF SPEC v4*